START HERE

Check one feature of your own app

You do not need to start by testing everything. Pick one feature that handles something private or valuable: a partner note, an invoice, a file or an account change.

1. Define the expected result. Account A can open its own record. Account A should not open the private record belonging to account B.

2. Run a bounded check. Use a test copy, fictional data and two ordinary accounts. Have your assistant show the request and the smallest result that proves what happened.

3. Repeat after a repair. Check the same forbidden action again, then check that the legitimate owner still has access.

Use STARTER-PROMPT.txt for a copy-ready template. Replace its placeholders with your own app and one test record. Give login details privately, outside public prompts, recordings and reports.

An assistant needs browser or HTTP tools to execute a web test. A written plan is not evidence that a request ran. A status code alone is not evidence that private data was returned.

The checklist on the following pages helps you choose what to examine next. A passed test covers the action you checked; it does not certify the whole app.

CHECKLIST A

The same records can leak elsewhere

- [ ] Private records: check lists and individual records with two separate accounts, including workspaces and linked invoices. The server must check who can read or change each one.
- [ ] Signed-out access: repeat protected-page and data requests without a login; they should not return private records.
- [ ] Search and exports: keep another account's notes, email addresses and rows out of results and downloaded spreadsheets.
- [ ] File links and storage buckets: a private attachment should stay private when its link is copied to a different account.
- [ ] Admin pages: hiding the menu is not enough. An ordinary account must also be blocked from the page's underlying requests.
- [ ] Profile fields and invitations: only allow fields the user can change; stop users granting roles or inviting people with powers they do not have.
- [ ] Database permissions: enforce access rules on every relevant table; limit what the app's database account can read or change.
- [ ] API responses: return only the fields the page needs. A hidden field in a response is still available to the browser.

CHECKLIST B

Getting in without the right login

- [ ] Sessions: check expiration, logout and revocation after a compromise. Let users see and end other sessions; closing a browser alone does not prove a session is broken.
- [ ] Password resets: use expiring, single-use links for the intended account. Avoid responses that reveal which email addresses have accounts.
- [ ] Passwords and guessing: remove defaults, use strong password hashing and unique admin passwords, add a second sign-in check, and limit repeated failures.
- [ ] Google or GitHub login: verify allowed return addresses and provider protections. Check account linking and email verification so someone cannot claim another person's identity.
- [ ] Redirect links: keep login links from bouncing people to an unapproved site that could impersonate yours.
- [ ] Session tokens and cookies: validate signatures and expiry, protect signing secrets and reject tampered tokens. Protect cookies from script access where appropriate and send them securely.
- [ ] Cross-site requests and CORS: check whether another website can trigger a sensitive action or read a signed-in user's data. Restrict trusted origins and protect state-changing requests.
- [ ] Clickjacking and browser headers: stop sensitive pages being framed inside another site, and check the relevant browser security headers.

CHECKLIST C

Someone else can spend your usage

- [ ] Private service keys: keep them out of browser downloads, public code, screenshots and exposed configuration files. A public client key is not automatically a secret.
- [ ] If a secret leaked: replace it, check what it could access and review its usage. Removing the visible copy does not invalidate a stolen key.
- [ ] AI chat, search and email: set enforced per-user limits and check access on the server, not just on the screen.
- [ ] Signup and file uploads: bound request frequency, file sizes and storage so abuse cannot fill your database or storage.
- [ ] Costs: use restricted credentials, quotas and the spending controls the provider actually enforces. An alert alone does not stop spending.
- [ ] Availability: use a small, agreed test load. Cache expensive repeated work where safe, but keep quotas and access checks; caching alone does not prevent abuse.
- [ ] Old test deployments and internal dashboards: check whether they are still public, with weaker protections or default credentials.

CHECKLIST D

A normal input can become an instruction

- [ ] Comments and profile text: make sure text is displayed safely. If you allow rich formatting, sanitize it; use browser protections as an additional layer.
- [ ] Search and database filters: keep user input separate from database commands with parameterized queries. An error message alone does not prove a data leak.
- [ ] File uploads: check the actual content, size and allowed type. Store uploads where they cannot execute as server code.
- [ ] Download paths: keep a requested filename from reaching files outside its allowed folder.
- [ ] Link previews and URL imports: limit what the server can fetch so a supplied address cannot reach private internal services.
- [ ] Conversion tools and shell commands: keep user input out of executable commands; use safer libraries and fixed allowed operations.
- [ ] Saved objects: do not rebuild arbitrary executable objects from untrusted serialized data. Validate the format and use safer parsers.

CHECKLIST E

The app can believe the wrong price

- ☐ Prices, discounts and totals: calculate them on the server and verify the paid amount before delivering the product.
- ☐ Paid features: check the account's entitlement on each protected operation. A payment screen alone does not enforce it.
- ☐ Payment notifications: verify the provider signature and process repeated notifications safely, without granting the same benefit twice.
- ☐ Orders and invoices: check who can read them and who can change amounts or payment details. A writable invoice is not automatically a completed transfer.
- ☐ Sensitive account changes: require appropriate reauthentication before changing payout or recovery details.
- ☐ Card data: use the payment provider's supported checkout; avoid collecting raw card details you do not need.
- ☐ Subscriptions: show the charge, renewal timing and trial terms clearly. Test cancellation, and review the current rules of the stores and payment providers you use.

CHECKLIST F

Your own AI can be another way in

- [ ] Prompt injection: treat instructions found in documents, emails and websites as untrusted content. Test with harmless markers and a controlled destination.
- [ ] Connected tools: give the agent only the data and actions it needs. Require review before sensitive sends, deletions or financial changes.
- [ ] Separate users: one person's conversation, retrieved documents or agent memory must not turn up in another account.
- [ ] Generated code: review changes to authentication, permissions and secret handling before shipping them.
- [ ] Dependencies: check the libraries the AI installed for known vulnerabilities, suspicious packages and unnecessary permissions; update or remove them as appropriate.
- [ ] AI responses that trigger actions: validate the proposed action in ordinary code before executing it. A confident answer is not permission.

CHECKLIST G

Would you notice, and could you recover?

- ☐ Audit logs and monitoring: record important sign-ins, permission changes and unusual access; send alerts someone will actually review.
- ☐ Sensitive logs and source maps: check what you expose in diagnostics. Keep secrets and private records out of public errors, logs and published debugging files.
- ☐ Backups: keep recoverable copies with separate protection, then test a restore. A backup that has never been restored is an untested recovery plan.
- ☐ Incident response: have a way to pause a compromised feature or revoke its access while you investigate; test who can do that.
- ☐ Sensitive data: collect only what you need and protect it in transit and storage. Encryption does not replace permission checks.
- ☐ Privacy, cookies and retention: match your notices and choices to what the app collects, shares and keeps; get appropriate advice for your market.
- ☐ Package licenses: check the terms of third-party code before distributing it. A security scan does not decide your legal obligations.

The starter prompt

I own the app at YOUR_APP_URL. Test only that app and the backend services I explicitly identify. Use a test copy with fictional data.

I will provide two ordinary test accounts privately. Start by confirming what each can read normally. Then test one feature: can account A read account B's private TEST_RECORD using the same link or request?

Show the request with credentials removed and the smallest result needed to prove what happened. A successful status code is not proof that private data was returned. If a request fails to run, say so.

Do not edit records, change permissions, guess passwords, process payments, send messages or test unrelated sites. Stop after six explicit test requests beyond normal page loading, or sooner once this one check is resolved.

Report the expected behavior, the observed behavior and the evidence. If access is blocked, treat that as the result rather than forcing a finding. After I approve and apply any fix, repeat the same check and verify that normal owner access still works. Do not call the whole app secure from one passing test.

If the assistant cannot reach the app or use its tools, fix that setup problem before interpreting the result. Never count a tool failure as proof of protection.

RESULT WORKSHEET

Keep the evidence and retest

Feature and test URL

Account allowed to use it

Account that should be blocked

Exact action or request, with secrets removed

Expected result

Observed result and evidence

Repair made

Same test after repair

Normal owner action still works

Not tested or still uncertain